AI ASSISTANT CODING

ASSIGNMENT - 21.3

Name: Saketh Birru

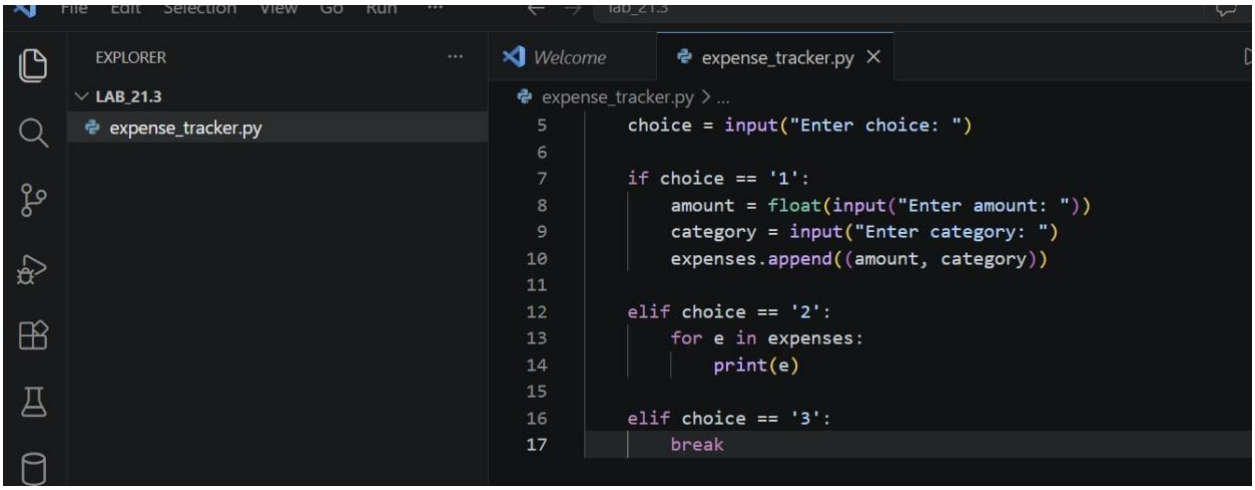
Ht.no: 2303A51403

Batch: 06

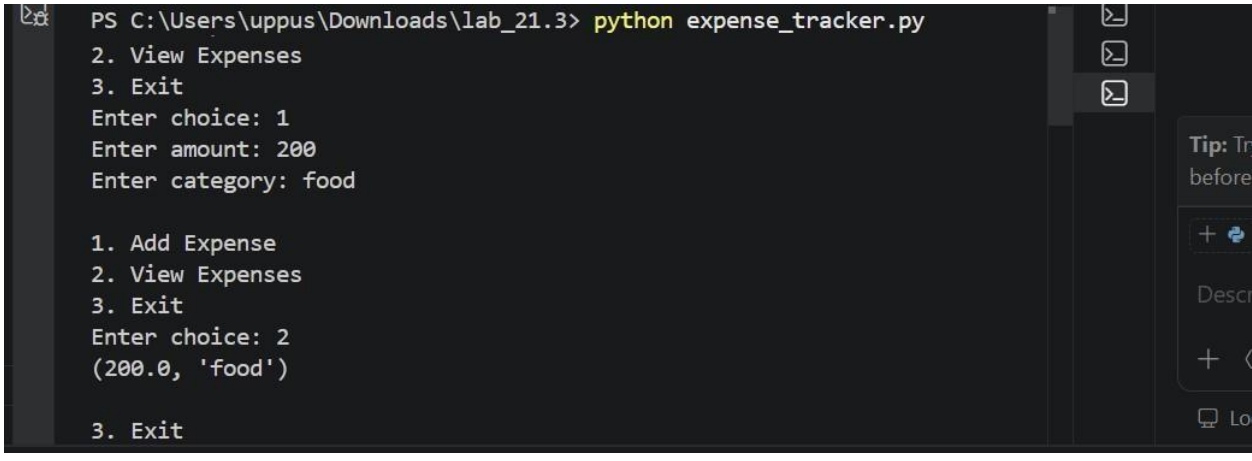
Task 1: Ai-Assisted Expense Tracker Application using Python

Prompt: Generate a basic Python program to record daily expenses with amount and category.

Then improve it by adding monthly summary, input validation, and modular functions Code:

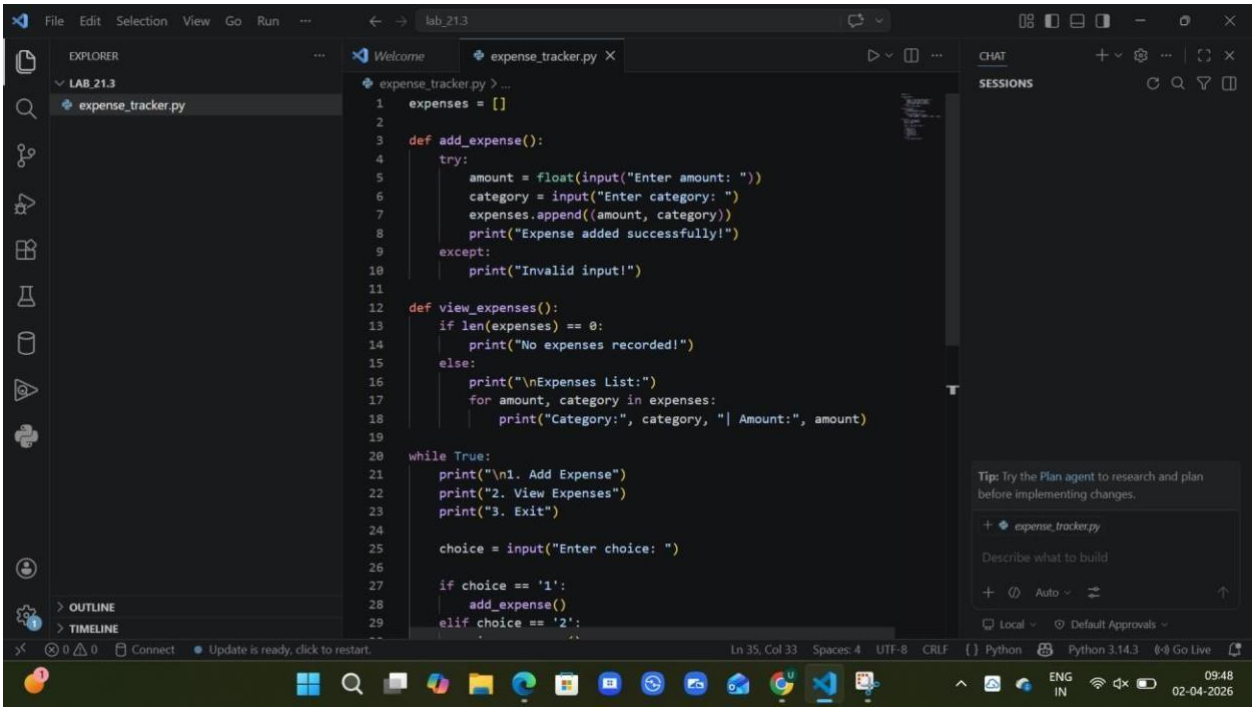


Output:



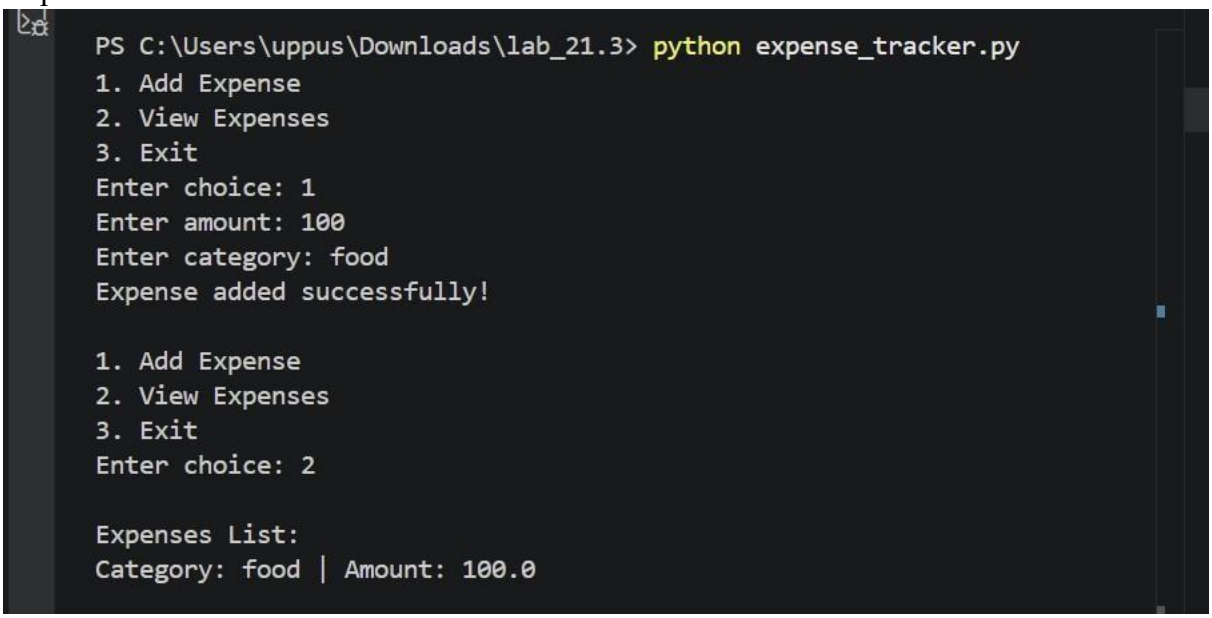
- The program stores expenses in a list
- User can:

- Add expense (amount + category)
- View expenses Corrtected Code:



```
expenses = []

def add_expense():
    try:
        amount = float(input("Enter amount: "))
        category = input("Enter category: ")
        expenses.append((amount, category))
        print("Expense added successfully!")
    except:
        print("Invalid input!")

def view_expenses():
    if len(expenses) == 0:
        print("No expenses recorded!")
    else:
        print("\nExpenses List:")
        for amount, category in expenses:
            print("Category:", category, "| Amount:", amount)

while True:
    print("\n1. Add Expense")
    print("2. View Expenses")
    print("3. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        add_expense()
    elif choice == '2':
```

Output:



```
PS C:\Users\uppus\Downloads\lab_21.3> python expense_tracker.py
1. Add Expense
2. View Expenses
3. Exit
Enter choice: 1
Enter amount: 100
Enter category: food
Expense added successfully!

1. Add Expense
2. View Expenses
3. Exit
Enter choice: 2

Expenses List:
Category: food | Amount: 100.0
```

Explanation:

- Code is divided into functions:
- `add_expense()` → adds expense
- `view_expenses()` → displays expenses
- Added **input validation using try-except**

- Displays **user-friendly output format**
- Shows message when no data is available

Task 2: Version Control Integration using Git – Student Grade Calculator

Prompt: Create a Python program to calculate average marks. Then enhance it using AI by adding grade classification and percentage calculation, and manage the project using Git with branching and merging."

Code: Initial Code

```
task2.py > ...
1   marks = [80, 75, 90]
2
3   avg = sum(marks) / len(marks)
4   print("Average:", avg)
```

Output:

```
PS C:\Users\uppus\Downloads\lab_21.3> python task2.py
PS C:\Users\uppus\Downloads\lab_21.3> python task2.py
Average: 81.66666666666667
PS C:\Users\uppus\Downloads\lab_21.3>
```

Explanation:

- Stores marks in a list
- Calculates average using `sum()` and `len()`
- Displays only average Corrected Code:

```python
marks = []

n = int(input("Enter number of subjects: "))

for i in range(n):
    m = int(input("Enter marks: "))
    marks.append(m)

avg = sum(marks) / len(marks)
percentage = avg

if percentage >= 90:
    grade = 'A'
elif percentage >= 75:
    grade = 'B'
elif percentage >= 50:
    grade = 'C'
else:
    grade = 'Fail'

print("Average:", avg)
print("Percentage:", percentage)
print("Grade:", grade)
```

Output:

```
PS C:\Users\uppus\Downloads\lab_21.3> python task2.py
Enter number of subjects: 2
Enter marks: 60
Enter marks: 95
Average: 77.5
Percentage: 77.5
Grade: B
PS C:\Users\uppus\Downloads\lab_21.3>
```

Explanation:

- Takes **user input** for marks
- Calculates: ○ Average ○ Percentage
- Assigns grade based on percentage
- Uses conditional statements for classification
- More realistic and user-friendly

Task 3: Password Validation Module with AI-Generated Commit Messages

Prompt: Create a Python program to validate a password with rules like minimum length and special character requirement. Also generate meaningful commit messages for the updates.

Code: Initial Code

```python
password = input("Enter password: ")

if password == "1234":
    print("Valid")
else:
    print("Invalid")
```

Output:

```
PS C:\Users\uppus\Downloads\lab_21.3> python task3.py
Enter password: 1234
Valid
PS C:\Users\uppus\Downloads\lab_21.3> python task3.py
Enter password: sony@0610
Invalid
PS C:\Users\uppus\Downloads\lab_21.3>
```

Explanation:

- Takes password input from user
- Checks only one fixed password ("1234")
- Prints valid or invalid

Corrected Code:

task3.py > ...

```python
import re

password = input("Enter password: ")

if len(password) < 6:
    print("Password too short!")
elif not re.search("[@#$%^&*]", password):
    print("Password must contain special character!")
else:
    print("Valid password")
```

Output:

```
PS C:\Users\uppus\Downloads\lab_21.3> python task3.py
Enter password: Sony@1234
Valid password
PS C:\Users\uppus\Downloads\lab_21.3> 1234
1234
PS C:\Users\uppus\Downloads\lab_21.3>
```

```
PS C:\Users\uppus\Downloads\lab_21.3> git config --global --list
PS C:\Users\uppus\Downloads\lab_21.3> git add .
>> git commit -m "Added password validation with minimum length and spec
ial character requirement"
[master (root-commit) c01d543] Added password validation with minimum le
ngth and special character requirement
 3 files changed, 68 insertions(+)
 create mode 100644 expense_tracker.py
 create mode 100644 task2.py
 create mode 100644 task3.py
```

Explanation:

- Uses **regular expressions (re module)** • Checks:
- Minimum length (6 characters)
- Special character requirement
- More secure and realistic
- Provides meaningful messages

Task 4: Quiz Management System using Pair Programming with AI Assistance and Git Integration

Prompt: Create a simple quiz program with questions and answers. Then enhance it using AI by adding score calculation and input validation. Simulate collaboration using Git branching and merging. Code: Initial Code

task4.py > ...

```python
questions = {
    "Capital of India?": "Delhi",
    "2+2?": "4"
}

for q in questions:
    ans = input(q + " ")
    if ans == questions[q]:
        print("Correct")
    else:
        print("Wrong")
```

Output:

```
PS C:\Users\uppus\Downloads\lab_21.3> python task4.py
Capital of India? Delhi
Correct
2+2? 4
Correct
PS C:\Users\uppus\Downloads\lab_21.3>
```

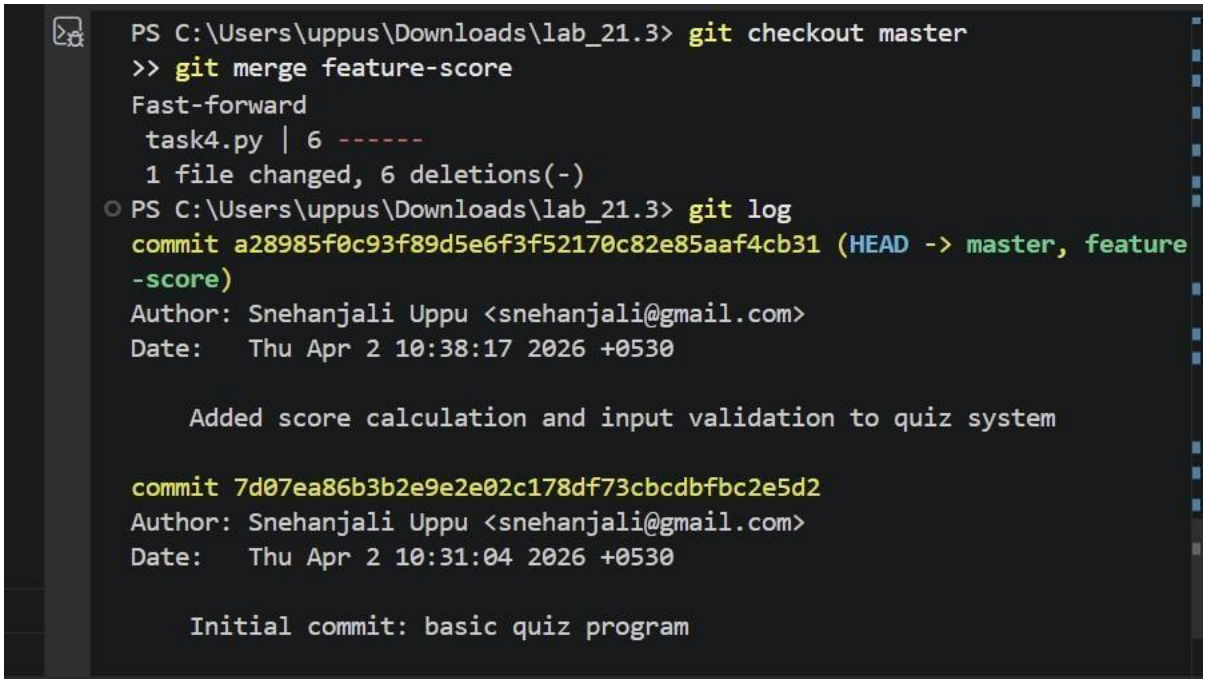
Explanation:

- tores questions and answers using a dictionary
- Loops through each question
- Takes user input
- Checks correctness and prints result Corrected Code:

```python
questions = {
    "Capital of India?": "Delhi",
    "2+2?": "4"
}

score = 0

for q in questions:
    ans = input(q + " ")

    if ans.lower() == questions[q].lower():
        print("Correct")
        score += 1
    else:
        print("Wrong")

print("Final Score:", score)
```

Output:

```
TERMINAL                                    powershell
PS C:\Users\uppus\Downloads\lab_21.3> python task4.py
Capital of India? Delhi
Correct
2+2? 4
Correct
Final Score: 2
PS C:\Users\uppus\Downloads\lab_21.3>
```



Explanation:
Basic quiz program was created (questions and answers)
Score calculation was added using AI assistance
Git was used to merge changes from branch to main